UNIVERSITY OF ABERDEEN

# JC4004 – Computational Intelligence

## Sessions 31 & 32: Vision transformers

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)
Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)
Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

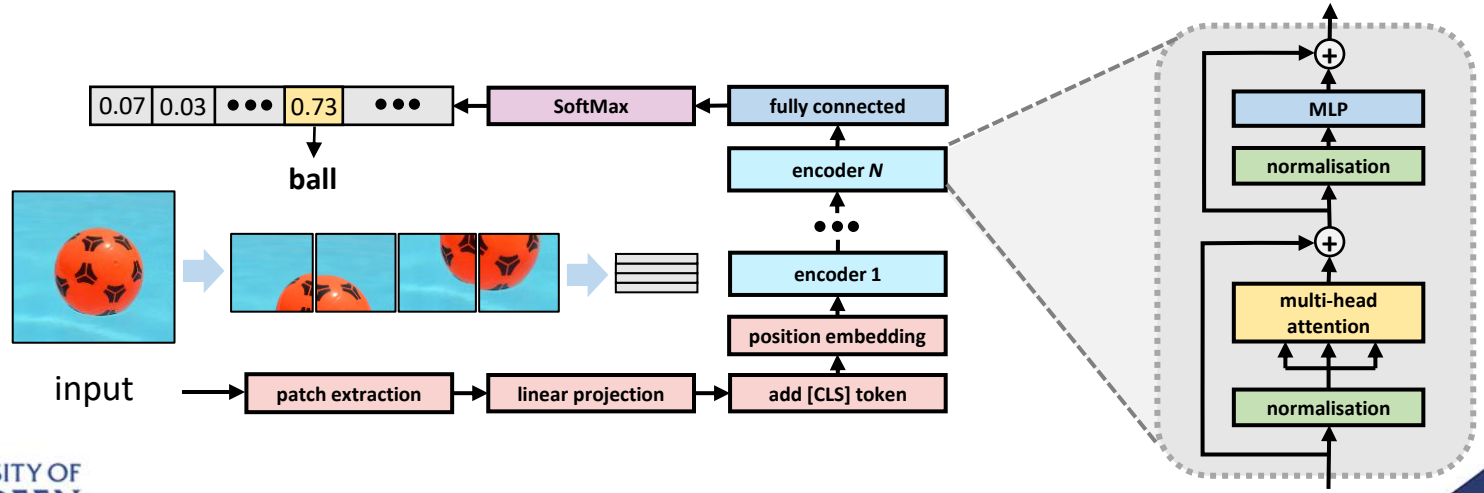
Oct-Dec 2025

# Transformers for computer vision?

- The huge success of transformers in NLP led many to wonder whether transformers could be useful in other types of applications as well
  - Computer vision is another area besides NLP where AI is applied excessively: how about employing transformers for computer vision?
- Computer vision applications are usually not dealing with sequences like NLP applications
  - Transformers for computer vision must reorganise visual data sequentially
  - One solution is to divide images into small patches and use a sequence of patches as input to a transformer model

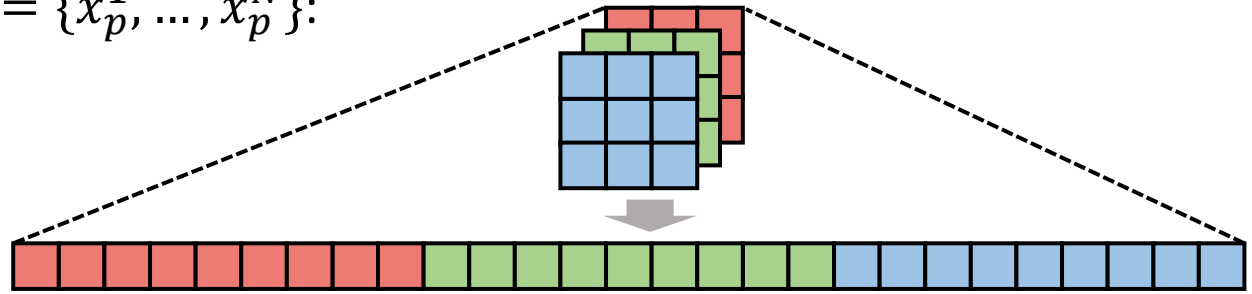
# Vision transformer

- The most widely adopted transformer architecture for computer vision is *Vision Transformer (ViT)* proposed by **Dosovitskiy et al.** in 2020
  - 2D image patches flattened to vectors and the sequence of vectors representing patches fed to transformer encoder



# Linear projection

- First, $N$ patches of size $P \times P$ pixels are flattened into 1D vectors $\boldsymbol{x}_p = \{x_p^1, \dots, x_p^N\}$:

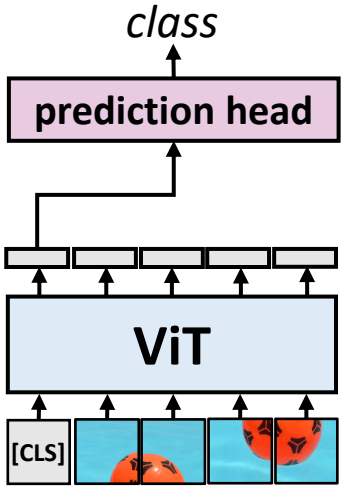


- Then, the input vectors are mapped into $D$-dimensional embeddings $\boldsymbol{z}_0$ using learnable matrix $\mathbf{E}$ and learnable 1D positional embeddings $\mathbf{E}_{pos}$:

$$\boldsymbol{z}_0 = \left[x_{class}, x_p^1\mathbf{E}, \dots, x_p^N\mathbf{E}\right] + \mathbf{E}_{pos}$$

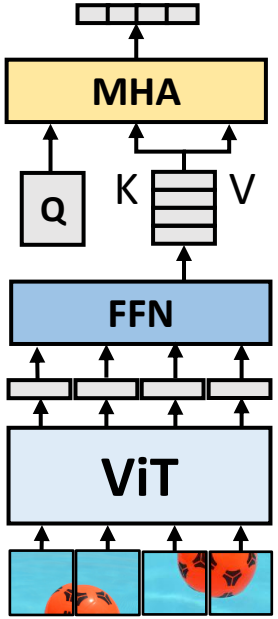
# Pooling

- Since transformer encoder gives a sequence of vectors as output, dummy [CLS] token can be added to the input sequence to represent the class
  - Usually, [CLS] is the first token (as in BERT)
  - The respective element in the output sequence used as input to the classifier
- Other pooling strategies have also been developed
  - *Global average pooling (GAP)* computes average for each element across the output sequence of vectors



# Multi-head attention pooling

- In consequent work, it was observed that removing the [CLS] token can save a lot of memory
  - Many hardware implementations pad sequences of tokens to multiples of 128
- *Multi-head attention pooling* can improve the performance in comparison to GAP
  - Feed-forward network reshapes sequence of vectors into a matrix used as $K$ and $V$ input to MHA layer
  - Learnable matrix used as $Q$ input to MHA layer
  - Short output vectors can be pooled by concatenation



# Original ViT variants

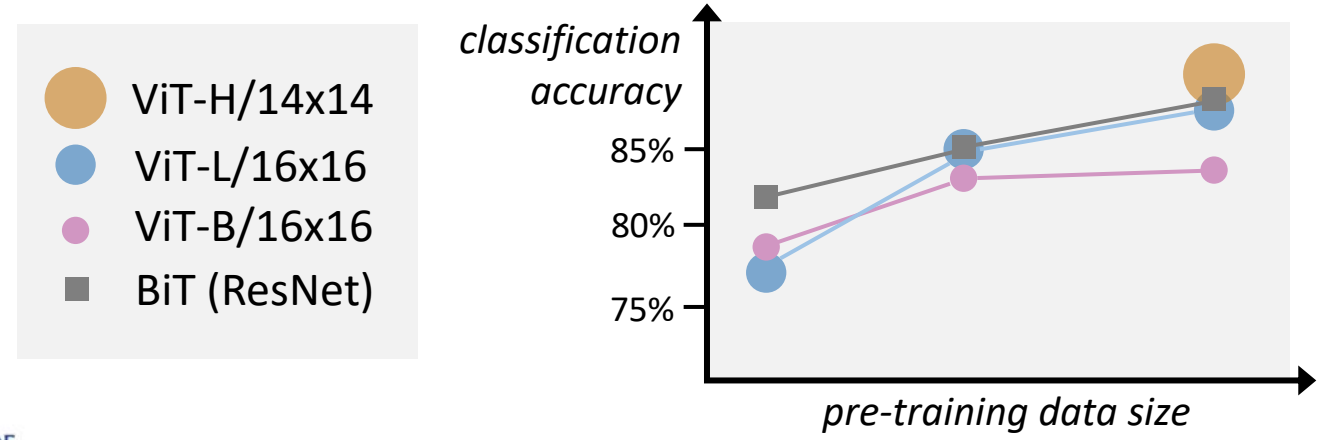
- The original paper presented several variants of ViT with different model sizes:

| Model | Layers | Hidden size *d* | MLP size | Heads | Parameters |
|---|---|---|---|---|---|
| ViT-Base | 12 | 768 | 3072 | 12 | 86M |
| ViT-Large | 24 | 1024 | 4096 | 16 | 307M |
| ViT-Huge | 32 | 1280 | 5120 | 16 | 632M |

- Best results achieved with patch sizes of $14 \times 14$ or $16 \times 16$ pixels

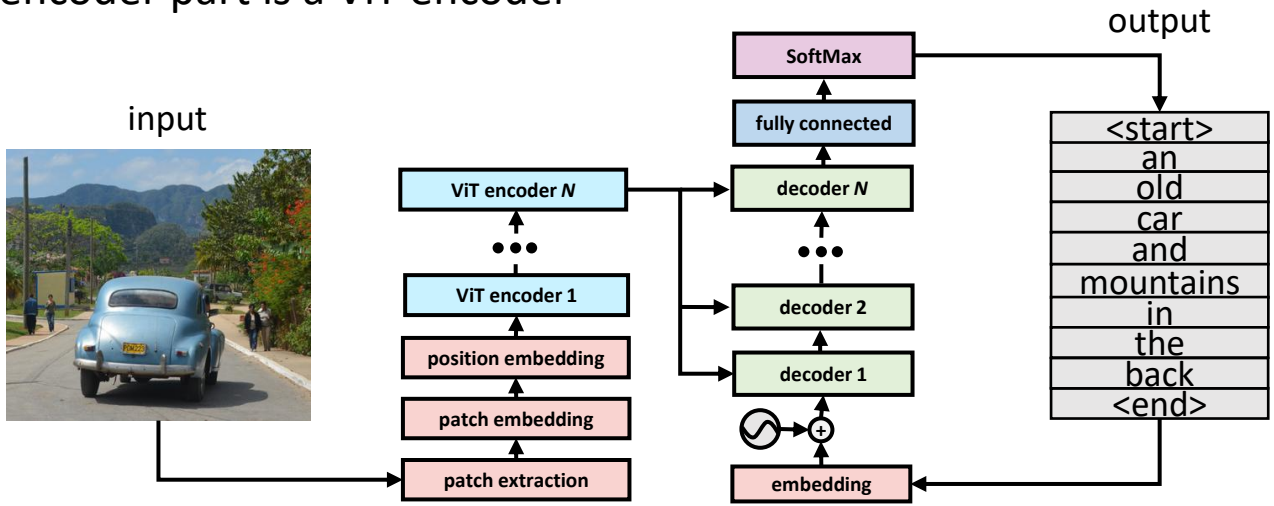
# Original ViT performance

- The experimental results indicate that ViT achieves better accuracy than ResNet models with similar computational budget
  - Larger models have better performance potential, but they also require larger amount of training data



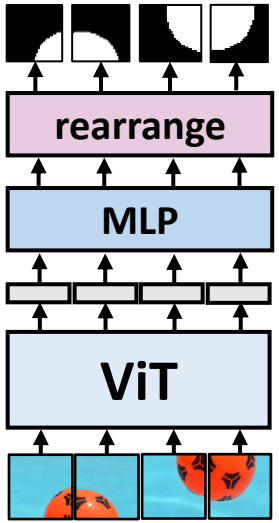
# ViT for image captioning

- Transformer encoder-decoder can be used for image captioning
  - Similar with basic transformer encoder-decoder model, except that the encoder part is a ViT encoder



# ViT for image segmentation

- We could use ViT also for image segmentation, as each output token corresponds to one input patch
  - For pixel-wise classification, we can use multilayer perceptron to convert each output vector to a vector of length $P \times P \times C$, where $C$ is the number of classes
  - The 1D vector can be converted to $C$ 2D patches of size $P \times P$ by rearranging the elements in the vector (inverse linear projection)
  - However, this is not the optimal method for image segmentation…

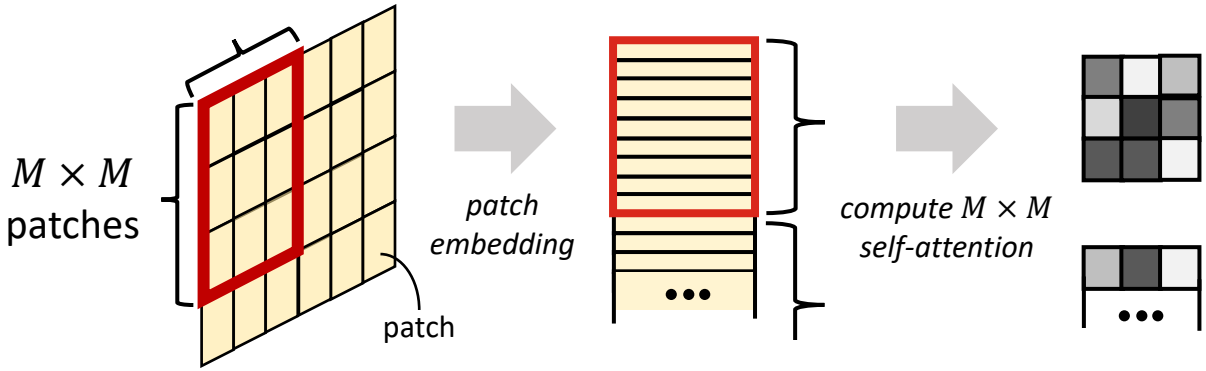


# Limitations of ViT

- Although ViT proved that transformers can be successfully applied to computer vision tasks, there are many limitations in the original ViT
  - For optimal performance, ViT requires more training data than CNNs
  - Computation of global attention has complexity $O(n^2)$, where $n$ is the number of patches: complexity is prohibitive for large resolutions
  - The vanilla ViT is not optimal for dense tasks like segmentation
- To alleviate the limitations, many improvements have been proposed
  - Hierarchical processing, local attention computation, etc.
  - We will discuss some of the proposed improved architectures like Swin-T

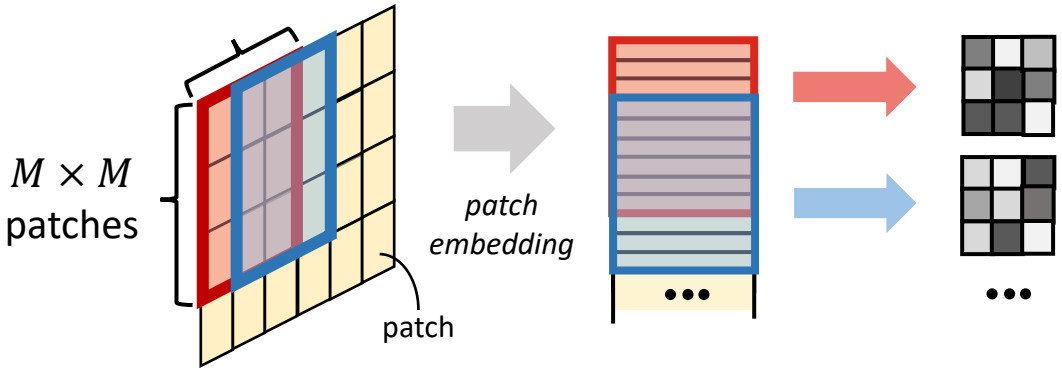
# Window-based self-attention

- Traditional self-attention is global, i.e., attention weights are computed between all the possible pairs of input tokens
- We can reduce complexity by *window-based self-attention:* attention computed for the patches (tokens) within non-overlapping windows



# Sliding window self-attention

- The problem of window-based self-attention with non-overlapping windows is that it ignores the connections between windows
- We can alleviate this problem by using sliding windows to compute self-attention, a little bit like sliding windows in convolution



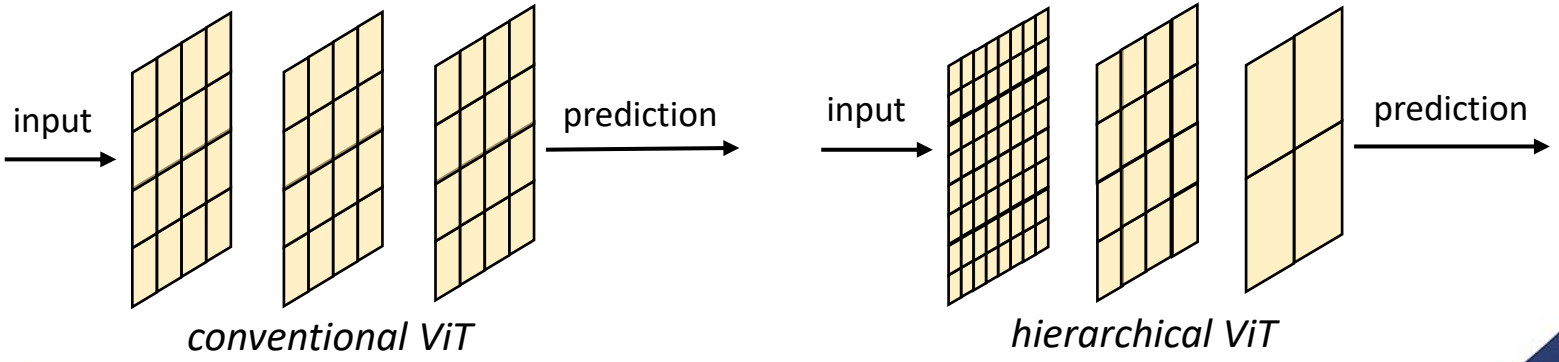
# Complexity comparisons

- Given that $n$ patches are divided into $r$ windows, complexity of windowed self-attention would be $O(r(n/r)^2) = O(n^2/r)$
- **Example:** for image of size $384 \times 384$ and patch size $16 \times 16$, there will be $24 \times 24 = 576$ patches and global self-attention complexity $O(576^2)$
  - If we use window-based local self-attention for windows of $4 \times 4$ patches, there will be $6 \times 6 = 36$ windows and complexity would be $O(576^2/36)$
  - If we use sliding window with stride $1$, we need to compute self-attention $21 \times 21 = 441$ times, so complexity is $O(441 \cdot 16^2) \approx O(0.34 \cdot 576^2)$

# Break: any questions?

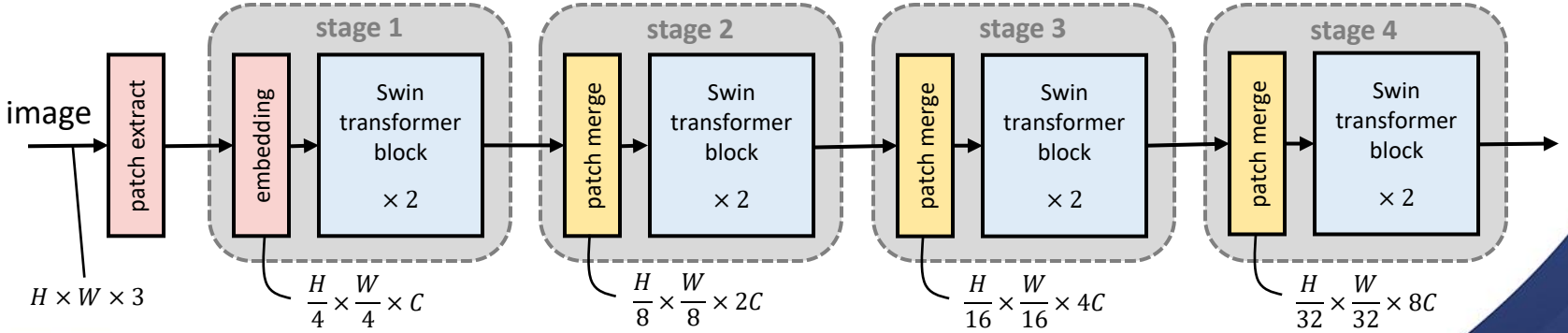
# Hierarchical processing

- In traditional transformers, the length of the sequence and the depth of the hidden layer remains the same from layer to layer
  - In computer vision, it can be beneficial to process patches hierarchically so that lower layers have higher resolution than deeper layers (like in CNNs)
  - Different methods for hierarchical processing in transformers proposed



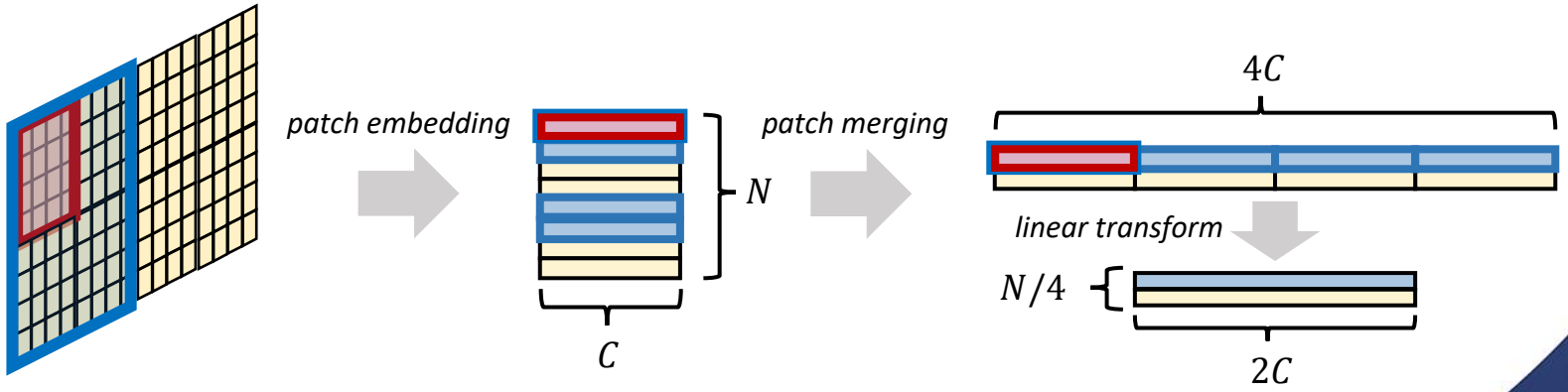
# Swin Transformer (Swin-T)

- *Shifted window transformer (Swin-T)* is one of the most prominent improved transformer architectures for computer vision
  - Processes data flow in a hierarchical manner via *patch merging*
  - Uses a combination of window-based and *shifted window self-attention* to reduce the complexity of self-attention computations



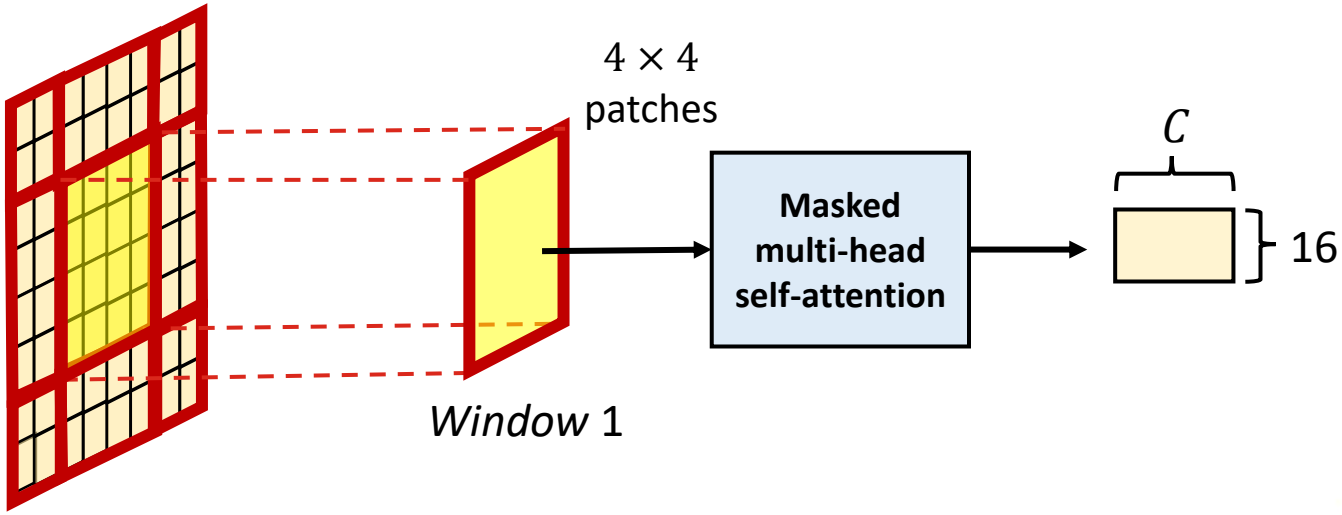
# Swin-T patch merging

- Swin-T patch merging process reduces spatial resolution in a somewhat similar manner as consecutive convolutions in CNNs
  - Embeddings representing patches to be merged are first concatenated
  - Linear transform (matrix multiplication) used to reduce channel dimension



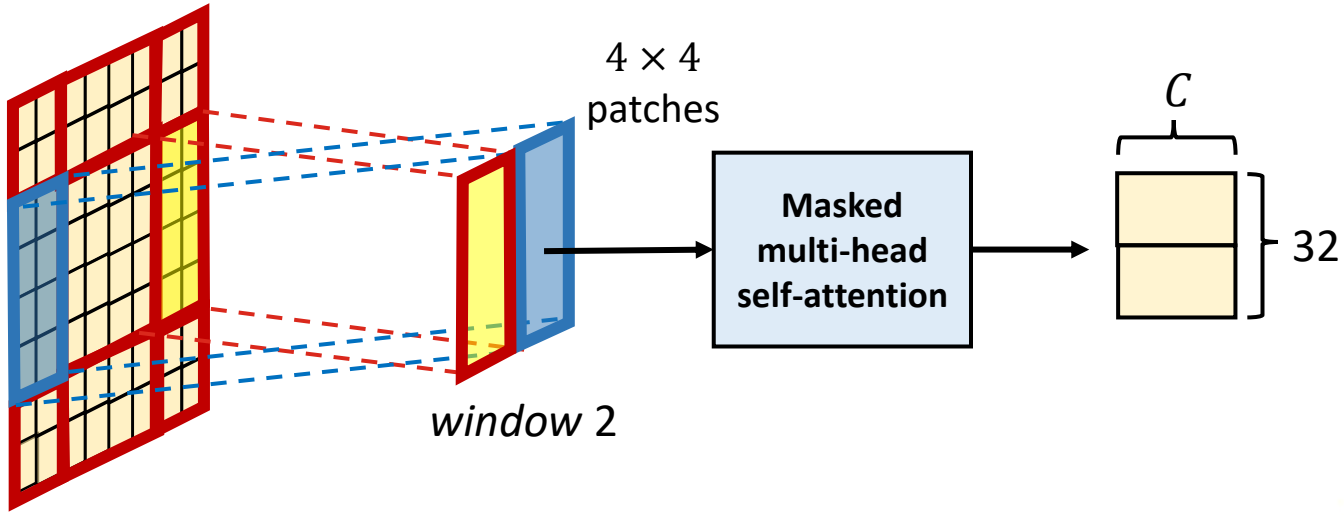
# Shifted window self-attention

- Swin-T introduces a concept of *shifted window self-attention*
  - Allows attention to consider connections across windows
  - Partitions windows by shifting patches to form overlapping areas



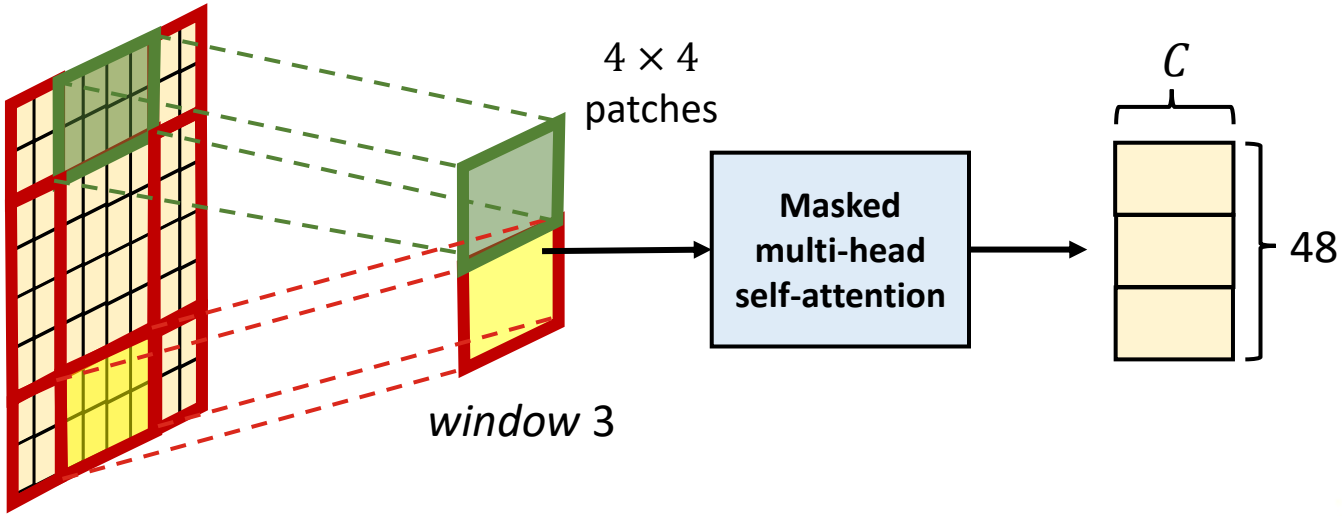
# Shifted window self-attention

- Swin-T introduces a concept of *shifted window self-attention*
  - Allows attention to consider connections across windows
  - Partitions windows by shifting patches to form overlapping areas



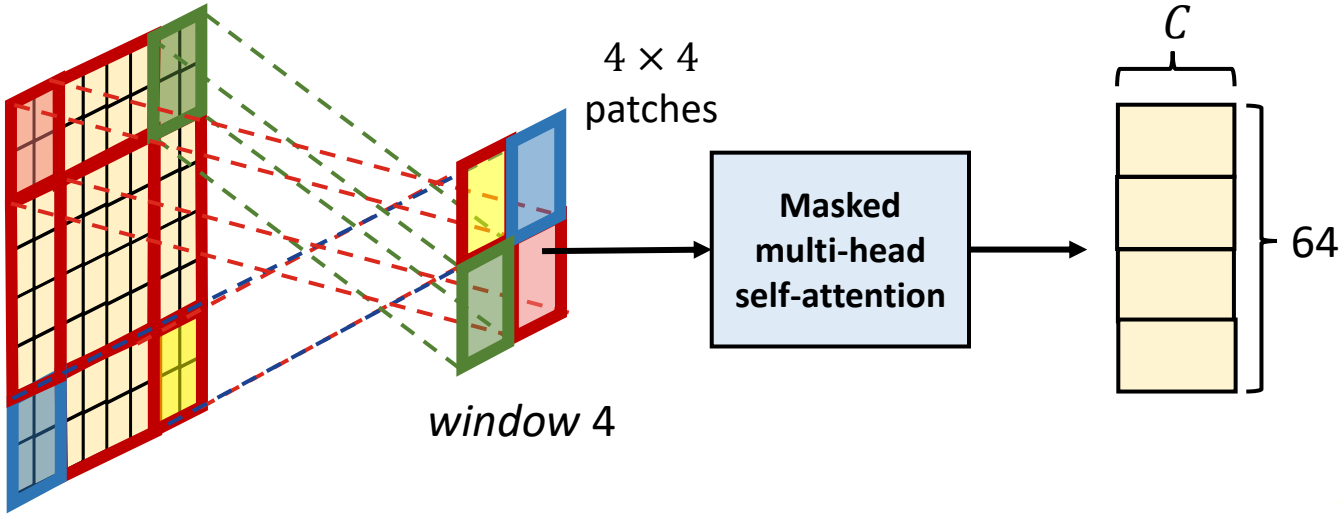
# Shifted window self-attention

- Swin-T introduces a concept of *shifted window self-attention*
  - Allows attention to consider connections across windows
  - Partitions windows by shifting patches to form overlapping areas



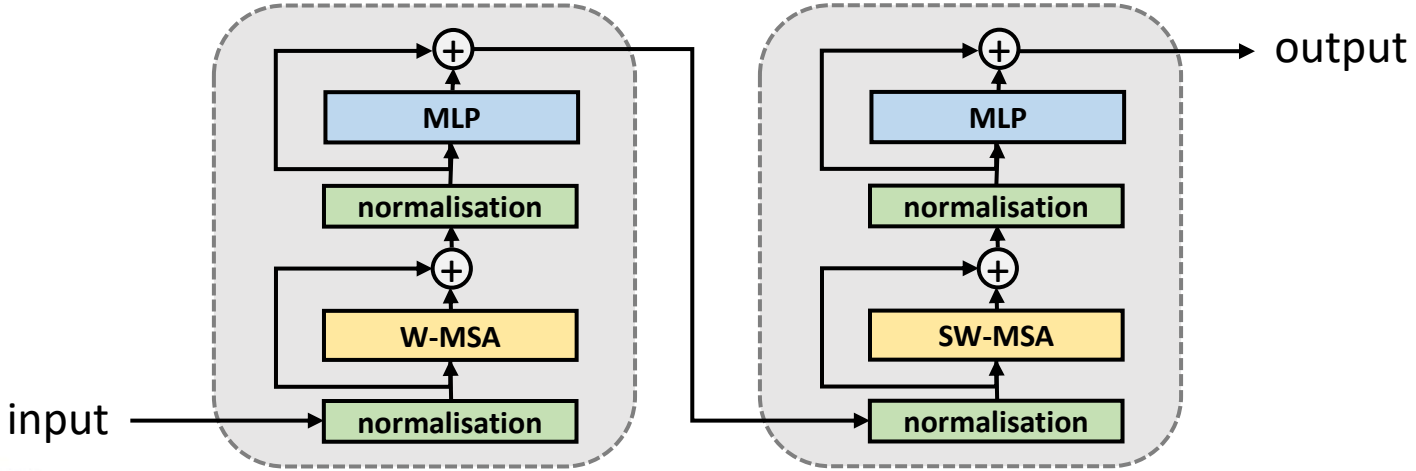
# Shifted window self-attention

- Swin-T introduces a concept of *shifted window self-attention*
  - Allows attention to consider connections across windows
  - Partitions windows by shifting patches to form overlapping areas



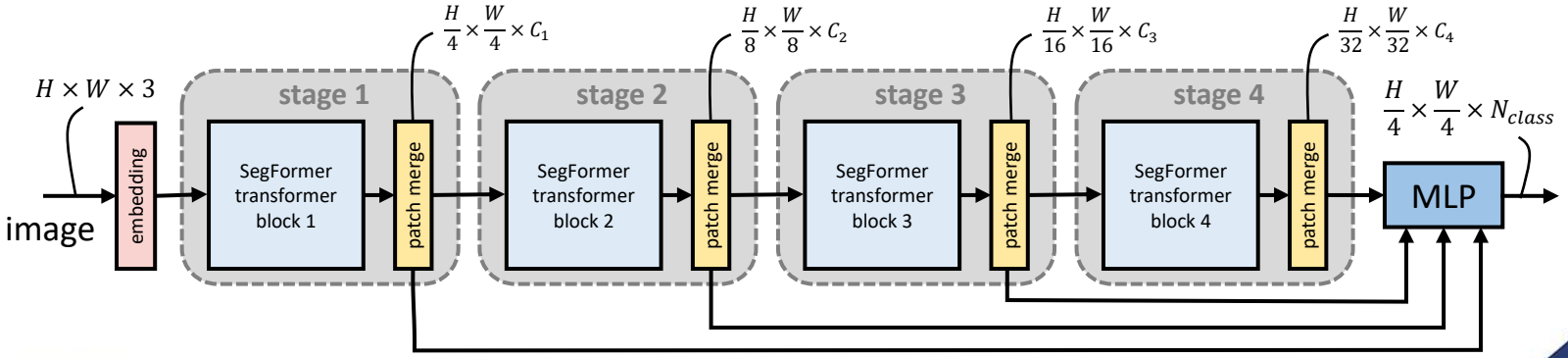
# Swin-T transformer blocks

- In Swin-T architecture, each stage combines two transformer blocks: the first with window-based multi-head self-attention (W-MSA), and the second with shifted window MSA (WS-MSA)



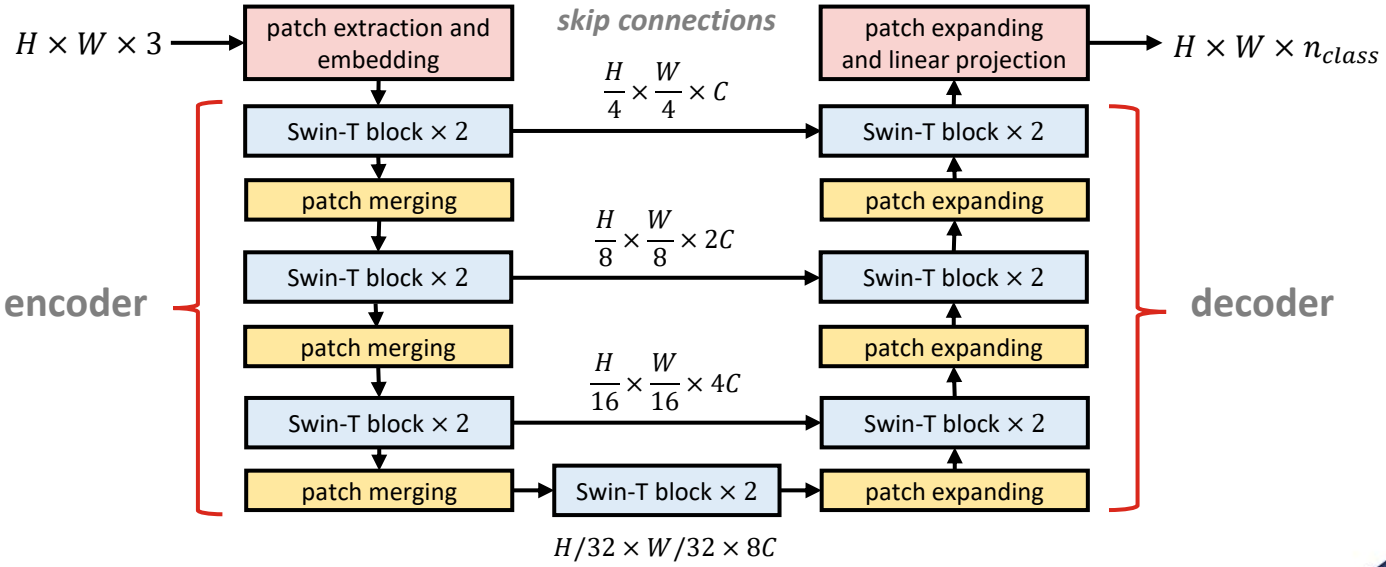
# SegFormer

- *SegFormer* is a transformer architecture designed specifically for semantic segmentation
  - Resembles Swin-T, but uses overlapping patches instead of shifted windows and combines patches from different stages to compute the class probability maps using multilayer perceptron for upsampling



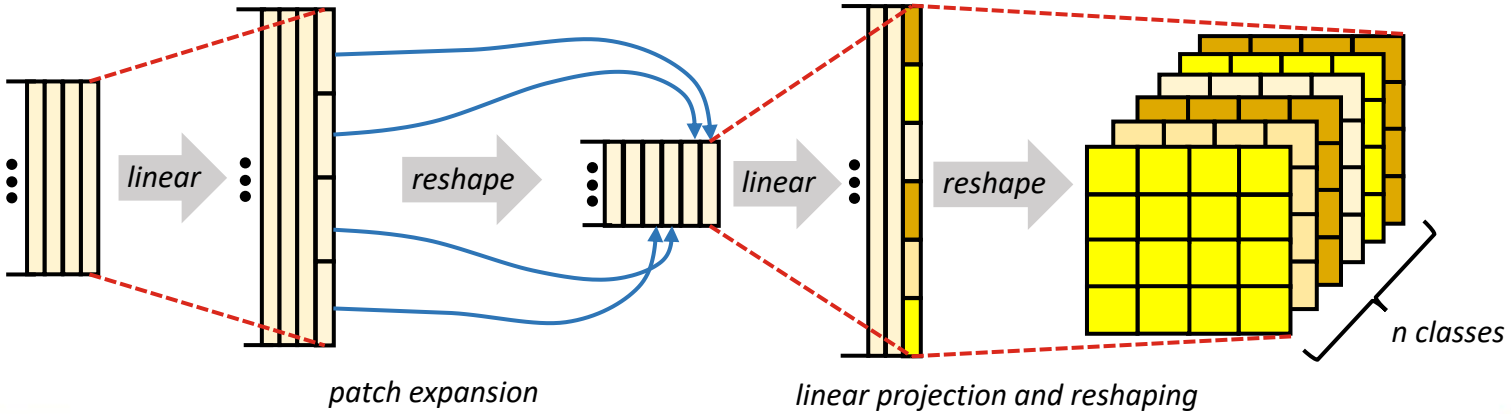
# Swin-U-Net

- U-Net architectures similar to CNN U-Nets can be constructed using Swin-T with patch expanding stages corresponding to patch merging



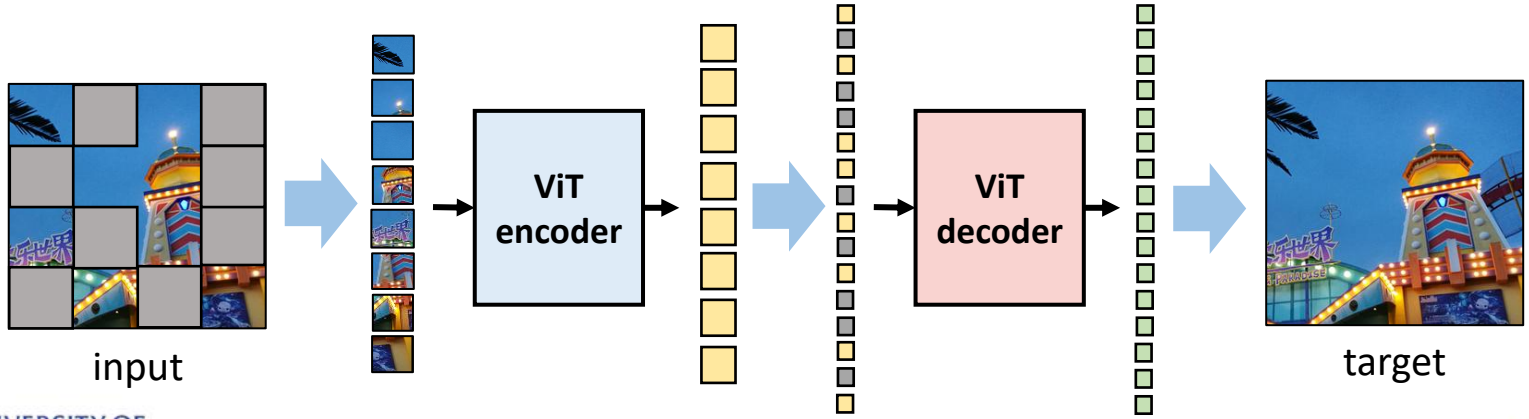
# Swin-U-Net patch expanding

- Patch expansion is essentially the inverse process of patch merging
  - Linear transforms used to change the lengths of the input vectors
  - Patch expansion by splitting vectors representing large patches into multiple shorter vectors representing the smaller patches



# Masked autoencoder

- Vision transformers typically pre-trained using supervised learning
- However, *masked autoencoders* have also shown good performance
  - Self-supervised learning by masking some of the input patches and reconstructing them from the signal (like pre-training BERT model)



# Summary

- *Vision Transformer (ViT)* was the first prominent transformer architecture designed for computer vision applications
  - Basic structure similar with transformer encoders such as BERT, except that the tokens represent non-overlapping patches extracted from an image
- Many improvements for ViT architecture have been proposed for more efficient computation and to support wider range of applications
  - Efficient self-attention: windowed, sliding window, and shifted window
  - Hierarchical processing of data via patch merging
  - Examples: Swin-T, SegFormer, Swin-U-Net, masked ViT autoencoder

# Questions and discussion